C Programming: Number Systems and Memory

Quick notes (sample)

1. Number systems

System	Base	Digits	Example
Binary	2	0, 1	11001 = 25
Octal	8	0 to 7	31 = 25
Decimal	10	0 to 9	25
Hexadecimal	16	0 to 9, A to F	0x2F = 47

Converting decimal to binary: divide by 2 repeatedly and read the remainders from bottom to top. For 25: 25/2=12 r1, 12/2=6 r0, 6/2=3 r0, 3/2=1 r1, 1/2=0 r1, so 25 = 11001.

2. Powers of two worth remembering

2^8 = 256, 2^10 = 1024 (1 KB), 2^16 = 65536, 2^20 = 1,048,576 (1 MB).

3. Negative numbers: two's complement

- Write the positive number in binary using the given number of bits.
- Invert every bit, then add 1.

Example (8 bits): 5 = 00000101. Invert: 11111010. Add 1: 11111011 (0xFB), which represents -5.

4. IEEE 754 single precision (32 bits)

Part	Bits	Meaning
Sign	1	0 = positive, 1 = negative
Exponent	8	Stored with bias 127
Mantissa	23	Fraction after the leading 1

Worked example: 5.75

```
5.75 = 101.11 in binary
    = 1.0111 x 2^2
exponent  = 2 + 127 = 129 = 10000001
mantissa  = 01110000000000000000000
bits      = 0 10000001 01110000000000000000000
hex       = 0x40B80000
```

5. Checking sizes in C

```
#include <stdio.h>
int main() {
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    return 0;
}
```

The size of int can differ between systems, so always check with sizeof.